Plugins

Quick Start

Plugins are just **small code modules** that extend OpenClaw with additional functionality (commands, tools, and Gateway RPC). Most of the time, you'll use plugins when you want a feature that isn't built into the core OpenClaw yet (or you want to keep optional features out of the main installation). Quick path:

1. See what's loaded:

```
openclaw plugins list
```

2. Install official plugin (e.g., Voice Call):

```
openclaw plugins install @openclaw/voice-call
```

3. Restart Gateway, then configure under `plugins.entries.<id>.config`.

Available Plugins (Official)

- From 2026.1.15, Microsoft Teams is only available as a plugin; if using Teams, install `@openclaw/msteams`.
- Memory (Core) — Bundled memory search plugin (enabled by default via `plugins.slots.memory`)
- Memory (LanceDB) — Bundled long-term memory plugin (auto-recall/capture; set `plugins.slots.memory = "memory-lancedb"`)
- Voice Call — `@openclaw/voice-call`
- Zalo Personal — `@openclaw/zalouser`
- Matrix — `@openclaw/matrix`
- Nostr — `@openclaw/nostr`
- Zalo — `@openclaw/zalo`
- Microsoft Teams — `@openclaw/msteams`
- Google Antigravity OAuth (provider auth) — Bundled as `google-antigravity-auth` (disabled by default)
- Gemini CLI OAuth (provider auth) — Bundled as `google-gemini-cli-auth` (disabled by default)
- Qwen OAuth (provider auth) — Bundled as `qwen-portal-auth` (disabled by default)
- Copilot Proxy (provider auth) — Local VS Code Copilot Proxy bridging; different from built-in `github-copilot` device login (bundled, disabled by default)

OpenClaw plugins are **TypeScript modules** loaded at runtime via jiti. **Configuration validation does not execute plugin code**; it uses the plugin manifest and JSON Schema. See Plugin Manifest. Plugins can register:

- Gateway RPC methods
- Gateway HTTP handlers
- Agent tools
- CLI commands
- Background services

- Optional configuration validation
- **Skills** (by listing `skills` directory in plugin manifest)
- **Auto-reply commands** (execute without calling AI agent)

Plugins run in the **same process** as Gateway, so treat them as trusted code. Tool writing guide: [Plugin Agent Tools](#).

Runtime Helpers

Plugins can access selected core helpers via `api.runtime`. For telephony TTS:

```
const result = await api.runtime.tts.textToSpeechTelephony({
  text: "Hello from OpenClaw",
  cfg: api.config,
});
```

Notes:

- Uses core `messages.tts` configuration (OpenAI or ElevenLabs).
- Returns PCM audio buffer + sample rate. Plugin must resample/encode for provider.
- Edge TTS is not supported for telephony.

Discovery and Priority

OpenClaw scans in order:

1. Configuration paths

- `plugins.load.paths` (files or directories)

1. Workspace extensions

- `<workspace>/.openclaw/extensions/*.ts`
- `<workspace>/.openclaw/extensions/*/index.ts`

1. Global extensions

- `~/.openclaw/extensions/*.ts`
- `~/.openclaw/extensions/*/index.ts`

1. Bundled extensions (shipped with OpenClaw, **disabled by default**)

- `<openclaw>/extensions/*`

Bundled plugins must be explicitly enabled via `plugins.entries.<id>.enabled` or `openclaw plugins enable <id>`. Installed plugins are enabled by default but can be disabled the same way. Each plugin must contain an `openclaw.plugin.json` file in its root directory. If the path points to a file, the plugin root is the file's directory and must contain the manifest. If multiple plugins resolve to the same id, the first match in the above order wins, and lower-priority duplicates are ignored.

Package Collections

Plugin directories can contain a `package.json` with `openclaw.extensions`:

```
{
  "name": "my-pack",
  "openclaw": {
    "extensions": ["./src/safety.ts", "./src/tools.ts"]
  }
}
```

Each entry becomes a plugin. If a package lists multiple extensions, the plugin id becomes `name/<fileBase>`. If your plugin imports npm dependencies, install them in that directory so `node_modules` is available (`npm install` / `pnpm install`).

Channel Directory Metadata

Channel plugins can broadcast onboarding metadata via `openclaw.channel` and installation hints via `openclaw.install`. This keeps the core directory data-free. Example:

```
{
  "name": "@openclaw/nextcloud-talk",
  "openclaw": {
    "extensions": ["./index.ts"],
    "channel": {
      "id": "nextcloud-talk",
      "label": "Nextcloud Talk",
      "selectionLabel": "Nextcloud Talk (self-hosted)",
      "docsPath": "/channels/nextcloud-talk",
      "docsLabel": "nextcloud-talk",
      "blurb": "Self-hosted chat via Nextcloud Talk webhook bots.",
      "order": 65,
      "aliases": ["nc-talk", "nc"]
    },
    "install": {
      "npmSpec": "@openclaw/nextcloud-talk",
      "localPath": "extensions/nextcloud-talk",
      "defaultChoice": "npm"
    }
  }
}
```

OpenClaw can also merge **external channel directories** (e.g., MPM registry exports). Place JSON files at one of:

- `~/.openclaw/mpm/plugins.json`
- `~/.openclaw/mpm/catalog.json`
- `~/.openclaw/plugins/catalog.json`

Or set `OPENCLAW_PLUGIN_CATALOG_PATHS` (or `OPENCLAW_MPM_CATALOG_PATHS`) to one or more JSON files (comma/semicolon/`PATH` separated). Each file should contain `{ "entries": [ { "name": "@scope/pkg", "openclaw": { "channel": {...}, "install": {...} } } ] }`.

Plugin IDs

Default plugin id:

- Package collection: `name` from `package.json`
- Standalone file: file basename (`~/.../voice-call.ts` → `voice-call`)

If a plugin exports `id`, OpenClaw will use it but will warn if it doesn't match the configured id.

Configuration

```
{
  plugins: {
    enabled: true,
    allow: ["voice-call"],
    deny: ["untrusted-plugin"],
    load: { paths: ["~/Projects/oss/voice-call-extension"] },
    entries: {
      "voice-call": { enabled: true, config: { provider: "twilio" } },
    },
  },
}
```

Fields:

- `enabled`: Main switch (default: true)
- `allow`: Allowlist (optional)
- `deny`: Denylist (optional; deny takes precedence)
- `load.paths`: Additional plugin files/directories
- `entries.<id>`: Per-plugin switches + configuration

Configuration changes **require Gateway restart**. Validation rules (strict):

- Unknown plugin ids in `entries`, `allow`, `deny`, or `slots` are **errors**.
- Unknown `channels.<id>` keys are **errors** unless plugin manifest declares channel id.
- Plugin configuration is validated using JSON Schema embedded in `openclaw.plugin.json` (`configSchema`).
- If a plugin is disabled, its configuration is preserved with a **warning**.

Plugin Slots (Exclusive Categories)

Some plugin categories are **exclusive** (only one active at a time). Use `plugins.slots` to choose which plugin owns the slot:

```
{
  plugins: {
    slots: {
      memory: "memory-core", // or "none" to disable memory plugins
    },
  },
}
```

If multiple plugins declare `kind: "memory"`, only the selected one loads. Others are disabled with diagnostics.

Control Interface (Schema + Labels)

Control interfaces use `config.schema` (JSON Schema + `uiHints`) to render better forms. OpenClaw enhances `uiHints` at runtime based on discovered plugins:

- Add per-plugin labels for `plugins.entries.<id>` / `.enabled` / `.config`

- Merge optional plugin-provided config field hints at: `plugins.entries.<id>.config.<field>`

If you want plugin configuration fields to display good labels/placeholders (and mark secrets as sensitive), provide `uiHints` and JSON Schema in your plugin manifest. Example:

```
{
  "id": "my-plugin",
  "configSchema": {
    "type": "object",
    "additionalProperties": false,
    "properties": {
      "apiKey": { "type": "string" },
      "region": { "type": "string" }
    }
  },
  "uiHints": {
    "apiKey": { "label": "API Key", "sensitive": true },
    "region": { "label": "Region", "placeholder": "us-east-1" }
  }
}
```

CLI

```
openclaw plugins list
openclaw plugins info <id>
openclaw plugins install <path>                 # copy a local file/dir into
~/.openclaw/extensions/<id>
openclaw plugins install ./extensions/voice-call # relative path ok
openclaw plugins install ./plugin.tgz           # install from a local tarball
openclaw plugins install ./plugin.zip           # install from a local zip
openclaw plugins install -l ./extensions/voice-call # link (no copy) for dev
openclaw plugins install @openclaw/voice-call # install from npm
openclaw plugins update <id>
openclaw plugins update --all
openclaw plugins enable <id>
openclaw plugins disable <id>
openclaw plugins doctor
```

`plugins update` only works for npm installations tracked under `plugins.installs`. Plugins can also register their own top-level commands (e.g., `openclaw voicecall`).

Plugin API (Overview)

Plugins export one of:

- Function: `(api) => { ... }`
- Object: `{ id, name, configSchema, register(api) { ... } }`

Plugin Hooks

Plugins can ship with hooks and register them at runtime. This lets plugins bundle event-driven automation without needing to install a separate hooks package.

Example

```
import { registerPluginHooksFromDir } from "openclaw/plugin-sdk";

export default function register(api) {
  registerPluginHooksFromDir(api, "./hooks");
}
```

Notes:

- Hook directories follow normal hook structure (`HOOK.md` + `handler.ts`).
- Hook eligibility rules still apply (OS/binary/environment/config requirements).
- Plugin-managed hooks appear as `plugin:<id>` in `openclaw hooks list`.
- You cannot enable/disable plugin-managed hooks via `openclaw hooks`; instead enable/disable the plugin.

Provider Plugins (Model Authentication)

Plugins can register **model provider authentication** flows so users can run OAuth or API key setup within OpenClaw (without external scripts). Register providers via `api.registerProvider(...)`. Each provider exposes one or more authentication methods (OAuth, API key, device code, etc.). These methods drive:

- `openclaw models auth login --provider <id> [--method <id>]`

Example:

```
api.registerProvider({
  id: "acme",
  label: "AcmeAI",
  auth: [
    {
      id: "oauth",
      label: "OAuth",
      kind: "oauth",
      run: async (ctx) => {
        // Run OAuth flow and return auth profiles.
        return {
          profiles: [
            {
              profileId: "acme:default",
              credential: {
                type: "oauth",
                provider: "acme",
                access: "...",
                refresh: "...",
                expires: Date.now() + 3600 * 1000,
              },
            },
          ],
          defaultModel: "acme/opus-1",
        };
      },
    },
  ],
});
```

Notes:

- `run` receives `ProviderAuthContext` with `prompter`, `runtime`, `openUrl`, and `oauth.createVpsAwareHandlers` helpers.
- Return `configPatch` when adding default model or provider configuration is needed.
- Return `defaultModel` so `--set-default` can update agent defaults.

Registering Message Channels

Plugins can register **channel plugins**, which behave like built-in channels (WhatsApp, Telegram, etc.). Channel configuration lives under `channels.<id>` and is validated by your channel plugin code.

```
const myChannel = {
  id: "acmechat",
  meta: {
    id: "acmechat",
    label: "AcmeChat",
    selectionLabel: "AcmeChat (API)",
    docsPath: "/channels/acmechat",
    blurb: "demo channel plugin.",
    aliases: ["acme"],
  },
  capabilities: { chatTypes: ["direct"] },
  config: {
    listAccountIds: (cfg) => Object.keys(cfg.channels?.acmechat?.accounts ??
{}),
    resolveAccount: (cfg, accountId) =>
      cfg.channels?.acmechat?.accounts?.[accountId ?? "default"] ?? {
        accountId,
      },
  },
  outbound: {
    deliveryMode: "direct",
    sendText: async () => ({ ok: true }),
  },
};

export default function (api) {
  api.registerChannel({ plugin: myChannel });
}
```

Notes:

- Put configuration under `channels.<id>` (not `plugins.entries`).
- `meta.label` is used for labels in CLI/UI lists.
- `meta.aliases` add alternate ids for normalization and CLI input.
- `meta.preferOver` lists channel ids to skip auto-enabling when both are configured.
- `meta.detailLabel` and `meta.systemImage` let UI display richer channel labels/icons.

Writing New Message Channels (Step-by-Step)

Use this when you want a **new chat interface** ("message channel") rather than a model provider. Model provider documentation is under `/providers/*`.

1. Choose id + configuration structure

- All channel configuration lives under `channels.<id>`.
- For multi-account setups, prefer `channels.<id>.accounts.<accountId>`.

1. Define channel metadata

- `meta.label`, `meta.selectionLabel`, `meta.docsPath`, `meta.blurb` control CLI/UI lists.
- `meta.docsPath` should point to a documentation page like `/channels/<id>`.
- `meta.preferOver` lets plugin replace another channel (auto-enable preferring it).
- `meta.detailLabel` and `meta.systemImage` are used by UI for detailed text/icons.

1. Implement required adapters

- `config.listAccountIds` + `config.resolveAccount`
- `capabilities` (chat types, media, threading, etc.)
- `outbound.deliveryMode` + `outbound.sendText` (for basic sending)

1. Add optional adapters as needed

- `setup` (wizard), `security` (DM policy), `status` (health/diagnostics)
- `gateway` (start/stop/login), `mentions`, `threading`, `streaming`
- `actions` (message actions), `commands` (native command behavior)

1. Register channel in plugin

- `api.registerChannel({ plugin })`

Minimal configuration example:

```
{
  channels: {
    acmechat: {
      accounts: {
        default: { token: "ACME_TOKEN", enabled: true },
      },
    },
  },
}
```

Load the plugin (extensions directory or `plugins.load.paths`), restart Gateway, then configure `channels.<id>` in your configuration.

Agent Tools

See dedicated guide: Plugin Agent Tools.

Registering Gateway RPC Methods

```
export default function (api) {
  api.registerGatewayMethod("myplugin.status", ({ respond }) => {
    respond(true, { ok: true });
  });
}
```

Registering CLI Commands

```
export default function (api) {
  api.registerCli(
    ({ program }) => {
      program.command("mycmd").action(() => {
        console.log("Hello");
      });
    },
    { commands: ["mycmd"] },
  );
}
```

Registering Auto-Reply Commands

Plugins can register custom slash commands that execute **without calling the AI agent**. This is useful for toggle commands, status checks, or quick operations that don't need LLM processing.

```
export default function (api) {
  api.registerCommand({
    name: "mystatus",
    description: "Show plugin status",
    handler: (ctx) => ({
      text: `Plugin is running! Channel: ${ctx.channel}`,
    }),
  });
}
```

Command handler context:

- `senderId`: ID of the sender (if available)
- `channel`: Channel where the command was sent
- `isAuthorizedSender`: Whether the sender is an authorized user
- `args`: Arguments passed after the command (if `acceptsArgs: true`)
- `commandBody`: Full command text
- `config`: Current OpenClaw configuration

Command options:

- `name`: Command name (without leading `/`)
- `description`: Help text shown in command list
- `acceptsArgs`: Whether command accepts arguments (default: false). If false and arguments are provided, command won't match and message will be passed to other handlers
- `requireAuth`: Whether authorized sender is required (default: true)
- `handler`: Function returning `{ text: string }` (can be async)

Example with authorization and arguments:

```
api.registerCommand({
  name: "setmode",
  description: "Set plugin mode",
  acceptsArgs: true,
  requireAuth: true,
  handler: async (ctx) => {
    const mode = ctx.args?.trim() || "default";
    await saveMode(mode);
    return { text: `Mode set to: ${mode}` };
  },
});
```

Notes:

- Plugin commands are processed before built-in commands and AI agents
- Commands are registered globally and work across all channels
- Command names are case-insensitive (`/MyStatus` matches `/mystatus`)
- Command names must start with a letter and can only contain letters, numbers, hyphens, and underscores
- Reserved command names (like `help`, `status`, `reset`, etc.) cannot be overridden by plugins
- Duplicate command registrations across plugins will fail with diagnostic error

Registering Background Services

```
export default function (api) {
  api.registerService({
    id: "my-service",
    start: () => api.logger.info("ready"),
    stop: () => api.logger.info("bye"),
  });
}
```

Naming Conventions

- Gateway methods: `pluginId.action` (e.g., `voicecall.status`)
- Tools: `snake_case` (e.g., `voice_call`)
- CLI commands: kebab or camel, but avoid conflicts with core commands

Skills

Plugins can ship with Skills in their repo (`skills/<name>/SKILL.md`). Enable it using `plugins.entries.<id>.enabled` (or other configuration gating) and ensure it exists in your workspace/hosted Skills location.

Distribution (npm)

Recommended packaging:

- Main package: `openclaw` (this repo)
- Plugins: Separate npm packages under `@openclaw/*` (e.g., `@openclaw/voice-call`)

Publishing contract:

- Plugin `package.json` must contain `openclaw.extensions` with one or more entry files.
- Entry files can be `.js` or `.ts` (jiti loads TS at runtime).
- `openclaw plugins install <npm-spec>` uses `npm pack`, extracts to `~/.openclaw/extensions/<id>/`, and enables it in configuration.
- Config key stability: Scoped packages are normalized to **scopeless** ids for `plugins.entries.*`.

Example Plugin: Voice Call

This repo contains a voice call plugin (Twilio or log fallback):

- Source: `extensions/voice-call`
- Skills: `skills/voice-call`
- CLI: `openclaw voicecall start|status`
- Tools: `voice_call`
- RPC: `voicecall.start`, `voicecall.status`
- Config (twilio): `provider: "twilio"` + `twilio.accountSid/authToken/from` (optional `statusCallbackUrl`, `twimlUrl`)
- Config (dev): `provider: "log"` (no network)

See [Voice Call] and `extensions/voice-call/README.md` for setup and usage.

Security Considerations

Plugins run in the same process as Gateway. Treat them as trusted code:

- Only install plugins you trust.
- Prefer using `plugins.allow` allowlist.
- Restart Gateway after changes.

Testing Plugins

Plugins can (and should) ship with tests:

- In-repo plugins can keep Vitest tests under `src/**` (e.g., `src/plugins/voice-call.plugin.test.ts`).
- Separately published plugins should run their own CI (lint/build/test) and validate `openclaw.extensions` points to built entry point (`dist/index.js`).

For more plugin content, refer to: [Plugins - OpenClaw]